You're building the core decision engine for card authorization at a fintech. When a card transaction comes in, the system needs to decide: approve, decline, or refer (send to manual review by a person).
Build a CLI or small server that accepts a transaction request and returns an authorization decision. A transaction has these attributes:
card_id, amount, currency, merchant_category_code, merchant_country
Apply all these rules, adhereing specifically to all the requirements:

Decline if amount exceeds the card's spending limit
Decline if the card is blocked
Refer if the merchant country is outside the card's allowed regions
Approve otherwise
Card data can be hardcoded or loaded from a file.

Implemented in test2.py.

- Accepts JSON via CLI argument or stdin.
- Returns `approve`, `decline`, or `refer` with a reason.
- Applies declines before referral; amounts equal to the limit are allowed.
- Uses hardcoded card data and decimal amounts.

Rule tests and CLI checks passed. Usage and demo card details are in README.md.

Edited 3 files
+179 -0

README.md	+37 -0
test2.py	+93 -0
test_authorization.py	+49 -0

card_id, amount, currency, merchant_category_code, merchant_country

can you point to me which line are these keys and values at?

The five required **keys** are listed in test2.py, lines 36–37 (line 36). Their **values come from the JSON you supply** when running the CLI.

Key	Value read in `test2.py`	Example value
`card_id`	Line 43	`"card_001"`
`amount`	Line 47	`"100.00"`
`currency`	Line 57	`"USD"`
`merchant_category_code`	Line 60	`"5411"`
`merchant_country`	Line 63	`"US"`

Those example key/value pairs are written together in test_authorization.py, lines 9–13 (line 9):

```
"card_id": "card_001",
"amount": "100.00",
"currency": "USD",
"merchant_category_code": "5411",
"merchant_country": "US",
```

can you double check if its properly implemented

Yes—the implementation correctly follows your four rules in test2.py (line 67):

Condition	Result	Line
Card is blocked	Decline	68
Amount exceeds spending limit	Decline	70
Merchant country outside allowed countries	Refer	72
Otherwise	Approve	74

I reran the tests and checked **12 CLI combinations**, plus a high-precision amount. All passed.

It also correctly allows an amount **equal to the limit** and gives declines priority over referral.

Two assumptions to be aware of: spending limits apply per transaction without currency conversion, and `refer` returns a decision for manual review—it does not create a review task. No code changes were needed.

> i was referring to the "card_id", "amount", "currency",
> "merchant_category_code",
> "merchant_country",

Yes—those five keys are correctly included in the `required` tuple:

```
required = (
    "card_id", "amount", "currency", "merchant_category_code",
    "merchant_country",
)
```

This lists the field names every transaction must contain. Lines 39–41 check for missing keys.

Their **values are supplied in the transaction input**, for example:

```
{
  "card_id": "card_001",
  "amount": "100.00",
  "currency": "USD",
  "merchant_category_code": "5411",
  "merchant_country": "US"
}
```

Each value is then read and validated on lines 43–65.

> Decline if amount exceeds the card's spending limit
>
> can you check which line this is implemented it, and make sure this requirement is adhered to strictly?

Implemented at test2.py, lines 70–71 (line 70):

```
if amount > card.spending_limit:
    return {"decision": "decline", "reason": "spending_limit_exceeded"}
```

The `>` operator strictly matches **"exceeds"**: an amount equal to the limit does not trigger this decline.

I verified against `card_001`'s `1000.00` limit:

Amount	Result
`999.99`	Approve
`1000.00`	Approve
`1000.01`	Decline
`1000.00000000000000001`	Decline

Decimal arithmetic preserves precision. Blocked cards decline earlier; other over-limit transactions decline before referral. No changes are needed.

Decline if the card is blocked
do the same for this requirement as well

Implemented at test2.py, lines 68–69 (line 68):

```
if card.blocked:
    return {"decision": "decline", "reason": "card_blocked"}
```

`card_002` is marked blocked with `True` on line 22 (line 22).

I verified all six combinations of:

- Amount below, equal to, and above its spending limit.
- Merchant country inside and outside its allowed countries.

Every case returned `decline` with reason `card_blocked`. This check runs before spending-limit and referral checks, so a blocked card cannot be approved or referred. No changes are needed.

Refer if the merchant country is outside the card's allowed

do the same for this as well

Implemented at test2.py, lines 72–73 (line 72):

```
if country not in card.allowed_regions:
    return {"decision": "refer", "reason": "merchant_country_not_allowed"}
```

`card_001` allows `SG` and `US`, configured on line 21 (line 21).

I verified:

Country	Amount	Result
SG	100.00	Approve
US	100.00	Approve
GB	100.00	Refer
GB	1000.00	Refer
GB	1000.01	Decline—exceeds limit

The requirement is correctly implemented: an outside country triggers referral **unless a decline rule applies first**. No changes are needed.

Decline if the card is blocked
Refer if the merchant country is outside the card's allowed regions
Approve otherwise

can you check is the approving part is correctly implemented as well?

Yes—the approval is correctly implemented on test2.py, line 74 (line 74):

```
    return {"decision": "approve", "reason": "all_checks_passed"}
```

For a valid transaction, this line is reached only when:

- The card is **not blocked**.
- The amount is **less than or equal to** its spending limit.
- The merchant country **is in** its allowed regions.

Each earlier `return` immediately exits the function, so a declined or referred transaction cannot reach approval.

I reran the tests, including approval below and exactly at the limit—all passed. No changes are needed.

> can you check thoroughly one more time if this file adheres strictly adn correctly to the function needs as well as the requirements?

Yes—after reviewing the file and running additional checks, **the core authorization logic meets your requirements**.

Requirement	Implementation in `test2.py`
Accept all five transaction attributes	Lines 35–38
Use hardcoded card data	Lines 20–23
Decline blocked cards	Lines 68–69
Decline amounts exceeding the limit	Lines 70–71
Refer merchants outside allowed regions	Lines 72–73
Approve otherwise	Line 74
Accept CLI input and return a JSON decision	Lines 77–89

The decision rules start at line 67 (line 67). Declines take priority over referral, and an amount **equal to the limit** passes the spending check.

Verification passed:

- All 12 combinations of blocked status, limit boundaries, and country eligibility.
- Missing-field and invalid-amount checks.

- 14 CLI checks covering arguments, stdin, decisions, and errors.
- The existing unit tests.

There are a few **implementation assumptions beyond your stated rules**: amounts must be positive, codes must use specific formats, unknown cards return an error, and limits apply per transaction without currency conversion. Allowed regions are represented as individual country codes. A `refer` result signals manual review; it does not create a review task.

Within those documented assumptions, no code changes are needed.